

Subscription

This task intends to consolidate all your previous knowledge acquired on React States, React Hooks, Firebase and React Routes. You will expand your DEV@Deakin web platform with user session management.

Complete the following:

1. Complete the Task Requirements.
2. You will find “Topic Videos and Demo Videos” of Weeks 2 - 8 on the SIT313 CloudDeakin site particularly useful as a reference for this task.
3. Create a new [GitLab](#) repository for this task. Ensure all files/resources created are committed to this repository.
4. Create a 3 – 6 minute video where you provide a walkthrough of your code and solution, demonstrating your understanding of the concepts. Upload your video to [Panopto](#).
5. Create a short report that includes:
 - a. Screenshots of your DEV@Deakin web platform.
 - b. Screenshots of your Firestore of saved users.
 - c. A link to your repository on GitLab.
 - d. A link to your video.
6. **Upload** to OnTrack and **engage with feedback** to get this complete!

Task Requirements

Before completing this task, ensure you have completed all previous Pass and Credit tasks, and that your DEV@Deakin web platform has all the previous requirements and design criteria implemented successfully.

1. GitLab repository

- Create a New Project called Task D1. Set the Visibility Level to **Private**.
- Once the project is created, under **Manage->Members** add the Unit Chair (Aaron Spence) and your OnTrack marking tutor **as a Reporter role**.
- Strictly do not submit your `node_modules` folder to your repository. To prevent this, create a `.gitignore` file, and add inside it `node_modules/` to prevent this folder from being tracked by Git.
- Commit all code and resources (e.g. images) that you develop for this task into this repository, with proper commit comments.

Failure to follow the above steps means your task cannot be marked as Complete.

2. User Login/Logout

In Task P5 we connected our React frontend to our ExpressJS backend. Continue with this capability in this task.

Using NextJS?

If you opted to build your web platform in NextJS, you may continue to use NextJS as both the frontend and backend (and do not need to use ExpressJS as your backend).

In Task P4 we added a Login and Registration feature. Depending on how you completed that task, your current DEV@Deakin web platform may not be tracking which user is currently logged in. If that is the case, add this capability to your web platform with the following process flow:

1. Frontend: User registers for an account, which sends entered details to the backend.
2. Backend: Validate received data. If invalid, return a message back to the frontend. If successful, hash the password and store user's details into your Firestore and return a message back to the frontend.

3. Frontend: When a user logs in to their account, send entered details to the backend. Backend confirms email and password are valid; if yes, generate a *signed* JWT and return to frontend.
4. Frontend: Manage a user's session by storing their JWT in [localStorage\(\)](#). It may be fruitful to utilise React hooks like *useContext* to help manage the user's state throughout your web platform.
5. Frontend: Clear the active JWT when a user chooses to logout.

If interested, you may implement an industry-standard authentication provider for secure and robust user session management – see the **Optional Technologies** section.

3. Pricing Page

Add a Pricing link to your navigation bar. On your Pricing page, users can see the different features offered between a Free and Paid subscription to your DEV@Deakin web platform. Be creative and list some 'features' and decide which are offered under a Free or a Paid plan. For example:

- Early access to Paid posts.
- Increased Posts allowed per month.
- Increased size limit allowed for uploading images, and so on.

We are not actually going to implement the listed features – they are for demonstration purposes only.

You can use your own creativity here on how you wish to display the above information. Of course, aim to keep good user-experience in mind. Furthermore, your solution must be at a Distinction-level; be sure to invest sufficient effort.

4. Upgrade Plan Feature

Add an Upgrade Plan button to your Pricing page. Your page should work like so:

- Only currently logged in users can activate the Upgrade Plan button.

- If the user is on a Free plan, they may proceed with the Upgrade Plan feature.
- If the user is on a Paid plan, they cannot activate the Upgrade Plan button (or they receive an alert/notification instead).

When the Upgrade Plan button is pressed, create a *modal* popup where a user can enter in their payment details, such as name and credit card details. If data validation is successful, upgrade the user to a Paid plan – be sure to save this to your Firestore.

Display somewhere to indicate that they are on a Paid plan.

Visit the **Optional NPM Packages** section for potential enhancements to your page.

Optional Technologies

We strongly encourage you to explore additional technologies that are often standard in industry, that may assist you in completing this task:

- Authentication Providers – Sophisticated third-party options exist to handle user authentication and authorisation, that are standard to use in industry. Can explore options such as [Clerk](#), [Firebase](#), or others.
- [Zod](#) – An excellent library for data validation, such as usernames, emails and passwords. Robust data validation skills are crucial in industry.
- UI component library – These come with many pre-made elements (buttons, sliders, tables) to speed-up development time, and to create a more consistent style across your website. Some recommended ones include [PrimeReact](#), [Shadcn](#), [Mantine](#), and many more.

Optional NPM Packages

There is a rich ecosystem of amazing packages to enhance our development. Here are some that may assist you:

- [React-Credit-Cards-2](#) – A component for capturing credit card details.
- [SweetAlert2](#) – Beautiful, responsive, easy-to-use user popups.
- [React-Toastify](#) – Nice and easy user notifications.

Know of some awesome NPM packages? Let us know in the SIT313 Teams channel!

Caution: Beware of malicious NPM packages!

Unfortunately, NPM packages are now an active area infiltrated by malicious actors. NPM packages may appear 'safe' at glance, but may have embedded malicious code. Before installing any NPM package, investigate whether it is from a trustworthy author(s).

You are responsible for your own cyber-safety.

Available Resources

We have many resources to assist in your learning journey – if you are ever unsure then please reach out to us, we are always happy to help.

- **The SIT313 Teams channel** – an active learning resource with weekly announcements from the Unit Chair, and regular discussions/assistance from the SIT313 teaching team and students.
- **Seminars** – you can attend both on-campus and online sessions, and join as many as you like each week. Your tutors will provide demonstrations and assistance to help guide your learning.
- **SIT313 CloudDeakin** – contains a large collection of text and video resources.
- **Announcements** - Keep an eye on your email and any announcements made in the SIT313 Teams channel.

Caution: Acceptable use of Artificial Intelligence

It is acceptable to use AI as a learning tool. However, you can **never** ask AI to write code to complete some/parts of this task for you: this is cheating because it does not demonstrate that you understand anything. Detection of inappropriate use of AI has a very real consequences, such as failing the task, or even failing the entire unit.